Take-Home Assignment: Conversational Intake for Insurance

Objective

Design and build a working prototype of a conversational agent that completes a structured insurance application through conversation.

Build a web application with a React frontend and a backend technology of your choice (e.g., Node.js/Express, Python/FastAPI, or equivalent). Explain your choice and the tradeoffs.

The interface is split. On one side, a chat with the agent. On the other, the auto insurance quote application itself — a real form, visible at all times. As the conversation proceeds, the agent fills the form in. The user watches it happen.

The user may also edit any field directly, at any time, without saying anything in the chat. The agent must respect those edits and must not silently overwrite them on a later turn.

The agent should maintain context across a multi-turn conversation, ask for the fields it still needs, and decline to guess at values it has not been given.

If the agent cannot complete the application reliably — the user is confused, the data conflicts, the request is outside what the form covers — it should ask for clarification or offer human assistance rather than inventing values.

Users must be able to request a human agent, with the conversation context and the partially completed application sent to **Slack** for the customer-service team. You can use a free Slack account and token.

The Form

Implement an auto insurance quote application with these fields. The exact schema is yours to define; this is the shape we will be looking at.

Applicant — full name, date of birth, email, garaging state.

Vehicle — year, make, model, estimated annual mileage, ownership status (owned / financed / leased).

Coverage — liability limit (enum), collision deductible (enum), optional roadside assistance.

Claims history — number of at-fault claims in the past five years, and the date of the most recent one.

Two rules in that schema matter more than the rest, because they are where a model-proposed value meets validation you own:

- The minimum acceptable **liability limit depends on the garaging state**. Pick two or three states and give them different minimums; the rest can share a default.
- The **most recent claim date is required if and only if** the claim count is greater than zero, and must fall within the past five years.

Every field is flat. There are no repeating groups — that is deliberate, so your time goes into the agent and not into array editing.

Design Decisions

Consider how to design the backend components for conversation orchestration, LLM interaction, form state, validation, integrations, and persistence.

Decide how the agent writes to the form. Does it return the whole document each turn, or field-level changes? What does the wire format look like, and how do you keep the model's output conformant to your schema?

Decide how a user's direct edit re-enters the agent's context on the next turn, and what the agent is permitted to do with a field the user has touched.

Define the states a field can be in — empty, proposed by the agent, confirmed, edited by the user, invalid — and how the interface communicates each one. A value the agent inferred and a value the user typed are not the same thing, and the user should be able to tell them apart.

Explain where validation runs, and what happens when the agent proposes a value that fails it. The model should not be the last line of defense for your own schema.

Explain how the system decides when to ask, when to infer from what it already has, and when to escalate to a human.

Consider security issues including PII — this form is almost entirely PII — prompt injection, rate limiting, secrets management, and abuse prevention.

In-memory session state is acceptable for the prototype. Describe in the README how conversation history, form state, and field provenance would be persisted in production, and what that changes.

Cloud Architecture

Assume the production application will run on **AWS, GCP, or Azure**, and choose the cloud services you consider appropriate. If you opt for a serverless architecture, explain why.

Describe how you would host the React application, backend APIs, database, session and form state, secrets, monitoring, and security controls.

Your architecture should be able to start small but scale to significant concurrent traffic without unnecessary infrastructure complexity.

Deliverables

Submit the source code, a README with local setup instructions, a high-level architecture diagram, and a short explanation of your key design decisions and tradeoffs.

The prototype should demonstrate at minimum:

1. **conversation → structured field updates → form reflects them**
2. **user edits a field directly → agent respects that edit on the next turn**
3. **user requests a human → Slack handoff carrying the conversation and the partially completed application**

Out of Scope

Please do not build these. They cost time and we are not evaluating them.

- Retrieval over a document corpus. There is no knowledge base in this exercise.
- Authentication, accounts, or multi-tenancy. A single anonymous session is fine.
- Durable persistence or session resume. In-memory is fine; reason about the production version in the README instead of building it.

- Real carrier or rating integrations. A stubbed premium estimate is fine.
- Repeating or nested field groups. The schema above is deliberately flat.
- A design system. Unstyled but legible beats polished but unfinished.

Notes

A **4–6 hour time box** is recommended, depending on your familiarity with the tools. Focus on sound architecture, engineering quality, and clear technical reasoning. **You may use AI-assisted coding techniques.**

We would rather see a few fields that round-trip correctly, with honest validation and a working handoff, than the full schema half-wired. If you run out of time, say so in the README and tell us what you would do next. Reserve the last hour for the written deliverables — the architecture diagram and your design reasoning are a substantial part of what we read.